AI Workforce OS

Project Documentation Report

An enterprise multi-agent platform: role-specialised AI employees that answer from company documents with citations and act on business tools, with permission checks, human approval for risky actions and a tamper-evident audit log.

Item	Detail
Course	Web Services and Service-Oriented Architecture (IT644), Autumn 2026
Version and date	Version 1, 11 October 2026
Repository	github.com/BitecodesHub/AI_Workforce_OS
Architecture file	docs/2026-10-11_Architecture_v1.md
Live deployment	13-55-126-54.sslip.io (recorded in the AWS deployment guide)
Prepared by	The AI Workforce OS team

Links open the public GitHub repository at the *main* branch. The architecture file and this PDF are new: their links work once both are committed and pushed.

Contents

Every entry is a link. Each page footer has a *Back to contents* link, and the PDF bookmarks panel lists every section.

Executive summary . . . 3

1. Project summary . . . 4

1.1 What the project does . . . 4

1.2 Main features . . . 4

1.3 Technologies . . . 4

1.4 How the parts work together . . . 5

2. Architecture overview . . . 6

2.1 Architecture file, section by section . . . 6

2.2 Services at a glance . . . 6

3. Project progress . . . 8

3.1 Completed . . . 8

3.2 Remaining . . . 8

4. Team members and roles . . . 11

5. Individual member documentation . . . 12

Member 1 — Aum Parmar . . . 12

Member 2 — Param Patel . . . 14

Member 3 — Love Shah . . . 16

Member 4 — Panthil Shah . . . 18

Member 5 — Fahim Saiyad . . . 20

Member 6 — Ismail Mansuri . . . 22

Member 7 — Yash Doshi . . . 24

Member 8 — Afif Momin . . . 26

6. About this report . . . 28

6.1 How the evidence was gathered . . . 28

6.2 Tests . . . 28

6.3 Plan task status . . . 28

6.4 Repository history notes . . . 28

6.5 Links in this document . . . 29

Executive summary

- **What it is.** AI Workforce OS lets a company run a team of AI employees (HR, Support, Research and others) from one Chat. They answer from the company's documents with citations and use business tools through connectors. Risky actions wait for a person's approval, and decisions are written to a hash-chained audit log.
- **How it is built.** Eight Spring Boot services (Java 21), five shared libraries and a React 19 web app with 27 page files. Each service owns its own PostgreSQL schema. It runs offline with a sandbox model, and is deployed on one AWS server that follows *main*.
- **Progress.** The core product is implemented: sign-in and roles, workspaces, agents, Chat, approvals, schedules, knowledge search, agent memory, 19 connector adapters, model fail-over, audit log, analytics and voice. Remaining work is mostly configuration (OAuth apps per provider), token revocation, knowledge-source crawlers, the unused event bus, and open items from the team contribution plan.
- **Team.** Eight members, with module ownership taken from the Team Contribution Plan and the README ownership table. The git history attributes almost all implementation commits to one account; see each member section and Repository history notes. Individual authorship marked *Needs confirmation* should be confirmed by the team.
- **Tests.** No tests were run for this report. The last recorded full run (6 October 2026) is quoted, with its source, in About this report.

1. Project summary

1.1 What the project does

- A company signs in, creates a workspace and gets a team of AI employees, each with a role, instructions, tools and a voice.
- People talk to the whole team in one **Chat**. Chat picks the right agent and says who took the work and why.
- Agents answer from the company's own documents (the **Knowledge** base) with citations, and act on business tools such as GitHub, Slack and Gmail through **connectors**.
- Every action is permission-checked. Sending, posting or deleting waits for a person to **approve** it. Decisions and outcomes go to a hash-chained **audit log**.
- AI models are configuration, not code: a workspace lists the models to use in order, and the router falls over to the next one when a model fails. With no API key at all, an offline sandbox model and practice data keep every screen working.

1.2 Main features

Feature	In plain words	Where
Workforce Chat	One conversation with every agent; @mentions, automatic routing, hand-offs between agents, inline approvals, clarifying questions	`orchestrator .../chat`
Orchestrator board	Live map of who is working on what; a "Needs you" inbox; pause or stop everything	`orchestrator .../board`
Agents and runs	Agents with versions, tool grants and approval rules; each run is saved step by step	`orchestrator .../service`
Schedules	Plain-English timetables ("every weekday at 9am")	`orchestrator .../schedule`
Knowledge	Upload documents; keyword plus meaning search; cited answers; each agent can have its own documents	`services/knowledge-service`
Memory	Agents remember and recall short notes; secrets are refused	`services/memory-service`
Connectors	19 business tools behind one tool gateway; live or practice data	`mcp-core`, `services/integrations-service`
Model routing	Seven AI providers plus an offline sandbox, with fail-over and usage accounting	`llm-core`
Identity and roles	Sign-in, sessions, five roles, permission codes, signed tokens	`services/identity-service`
Audit and analytics	Tamper-evident audit log with verify and export; spend and value figures	`services/analytics-service`
Voice	Speak to Chat and hear replies (ElevenLabs, or the browser's own speech)	`orchestrator .../voice`
Web app	27 page files including public /home and /trust pages	`web/src/routes`
Delivery	One-click launcher, Docker Compose, Kubernetes, CI, AWS deployment	`infra`, `.github/workflows`

1.3 Technologies

Layer	Technology
Backend	Java 21, Spring Boot 3.5, Spring Cloud 2025.0 (gateway), Maven (13 modules)
Web	React 19, TypeScript, Vite, TanStack Query, in-house router, Vitest
Data	PostgreSQL 17 with Flyway (one schema per service), Redis, Qdrant
AI models	OpenRouter, NVIDIA NIM, Groq, OpenAI, Anthropic, Google Gemini, AWS Bedrock, offline sandbox; Ollama as the last fallback on the AWS server

Layer	Technology
Delivery	Docker, Compose, Kubernetes (Kustomize), GitHub Actions, AWS CloudFormation, Caddy, Prometheus, Grafana

1.4 How the parts work together

A request travels through the system in five steps. The full diagrams are in the architecture file.



Figure 1. A chat request from input to output.

- **Identity** signs a short-lived token at sign-in; every service checks it itself.
- **Orchestrator** asks **org** for the AI provider key, **integrations** for connector tokens, **knowledge** for document passages and **memory** for notes.
- **llm-core** chooses the model; **mcp-core** checks and runs every tool call.
- Every service sends audit events to **analytics** through a local outbox, so none are lost.

2. Architecture overview

The separate architecture file holds the Mermaid diagrams, which GitHub renders: docs/2026-10-11_Architecture_v1.md. This section is a summary.

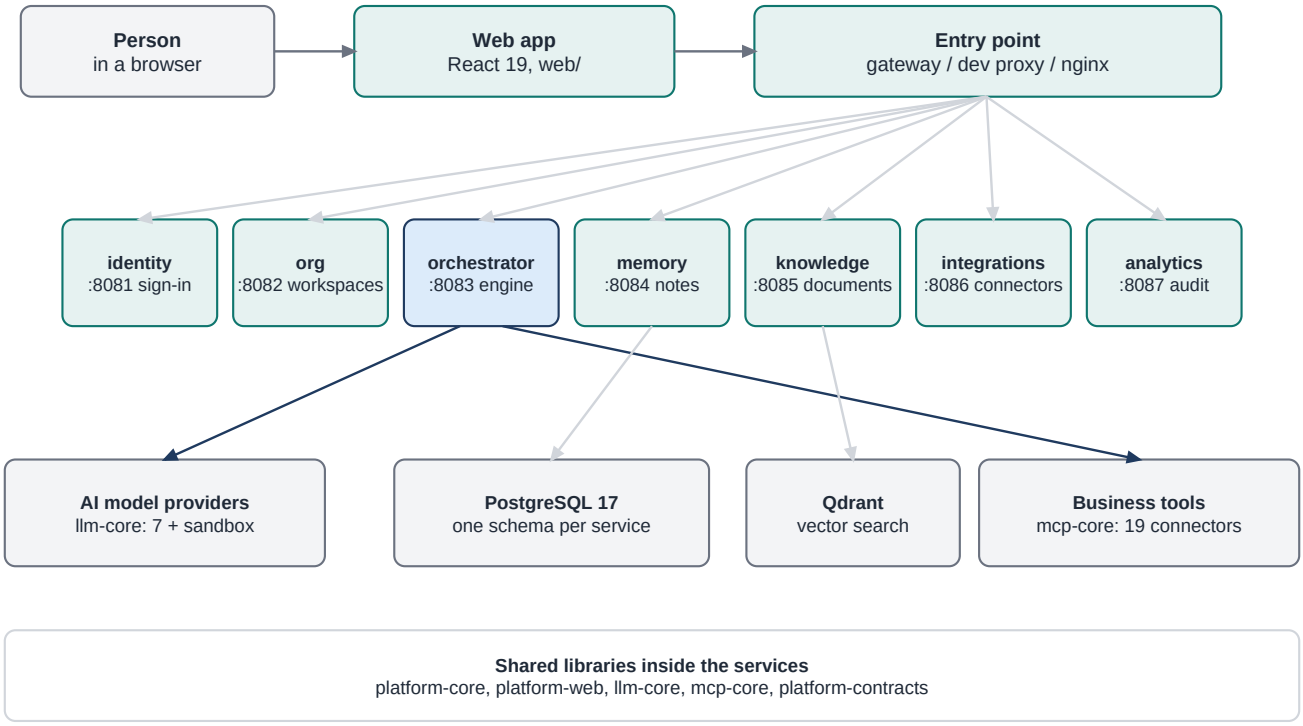


Figure 2. High-level architecture. The orchestrator (highlighted) is the engine; every service owns its own database schema.

2.1 Architecture file, section by section

Section (opens on GitHub)	What it shows	Related members
1. System overview	One diagram of the whole system	All
2. Components and responsibilities	What each service and library does	Aum Parmar, Param Patel
3. How components communicate	Who calls whom, tokens, audit outbox	Aum Parmar, Param Patel
4. Main data flow: a chat request	Sequence from typing a request to the answer	Love Shah, Panthil Shah, Yash Doshi
5. Tool calls and approvals	Tool gateway checks and the approval gate	Panthil Shah, Ismail Mansuri
6. Knowledge and memory	Ingestion and hybrid search	Love Shah, Fahim Saiyad
7. Technologies and integrations	Stack, providers, connectors	Ismail Mansuri, Afif Momin
8. Ways to run it	Dev, Compose, launcher, Kubernetes, AWS	Yash Doshi, Afif Momin

2.2 Services at a glance

Service	Port	Owns	Owner (from plan)
`gateway`	8080	Routing, token check, rate limits	Param Patel
`identity-service`	8081	Users, sessions, roles, tokens	Param Patel
`org-service`	8082	Workspaces, invitations, encrypted keys	Param Patel
`orchestrator-service`	8083	Agents, Chat, runs, approvals, schedules, model routing	Love Shah, Panthil Shah
`memory-service`	8084	Agent notes and run episodes	Love Shah

Service	Port	Owns	Owner (from plan)
`knowledge-service`	8085	Documents, embeddings, cited search	Fahim Saiyad
`integrations-service`	8086	Connector accounts and OAuth	Ismail Mansuri
`analytics-service`	8087	Audit log, dashboards	Not assigned in the plan

Note. analytics-service is not assigned to anyone in the plan or the README table. The audit-log screen (web) belongs to Yash Doshi's area. Owner: **Needs confirmation**.

3. Project progress

Two levels of evidence are used. **Code present**: the files were opened and implement the feature. **Reported verified**: a project document records a live test; it was not repeated for this report. Items that could not be settled from the repository are marked **Needs confirmation**.

3.1 Completed

	Area	What is done	Evidence
☑	Identity and access	Sign-in, sessions and devices, five roles, permission codes, ES256 tokens and JWKS; every service re-checks permissions	`identity-service` Param Patel
☑	Workspaces and secrets	Workspaces, invitations, envelope-encrypted API keys and tokens	`org-service` Param Patel
☑	Model routing	Seven providers plus a sandbox; ordered fail-over; usage and budget guard; budget cap endpoint	`llm-core .../router` `BudgetController`
☑	Agents and runs	Agent loop, approvals with real action class, questions, retry, send-back, tool-grant editing	`AgentRunner` Panthil Shah
☑	Chat, board, schedules	Routing with reasons, task graphs, background runs, Orchestrator board, plain-English schedules	`orchestrator .../chat` Love Shah
☑	Agent memory	Remember and recall during runs; curated notes; secrets refused	`memory-service` Love Shah
☑	Knowledge (RAG)	Upload, chunk, embed; hybrid cited search; per-agent documents	`knowledge .../service` Fahim Saiyad
☑	Connectors (MCP)	Tool gateway; 19 connectors with live and practice adapters; OAuth	`mcp-core` Ismail Mansuri
☑	Audit and analytics	Hash-chained audit log; verify on demand and nightly; export; dashboards	`AuditChainJob` `AuditController.java:213`
☑	Voice	ElevenLabs voice per agent; browser speech fallback	`orchestrator .../voice`
☑	Web app	27 page files, design system, 150 test files, accessibility audit (reported)	`web/src/routes` Yash Doshi
☑	Delivery	Docker, Compose, launcher, Kubernetes manifests, CI (4 jobs), AWS pull deployment with rollback	`infra` Afif Momin
☑	Documentation	README, code guide, test-and-fix plan (86 defects, all recorded as fixed), readiness log, AWS guide	`docs` Aum Parmar

3.2 Remaining

Gaps confirmed in the code or in project documents. Click a member name to jump to their section.

	Item	Status	Evidence	Area
☐	A1: the stack-change ADR does not name the old stack (FastAPI, Next.js), lists TanStack Router, which the web app does not use, and shares number 0001 with another ADR	**Partial**	`docs/adr/0001-record-stack-change.md`	Aum Parmar
☐	A2: confirm each requirement status in the design document against the code. The Word file is unchanged since it was added, so the check cannot be seen in git	**Needs confirmation**	`System Design Document v2`	Aum Parmar
☐	platform-contracts holds package folders only; there are no shared types yet	**Not done**	`platform-contracts`	Aum Parmar

	Item	Status	Evidence	Area
☐	README is out of date in places: it says agents do not use memory, five connectors have no live adapter and no endpoint sets a budget; the code now has all three	**Not done**	`README.md:167`	Aum Parmar
☐	Five ADR files (0001 to 0005 "week" decisions) hold one placeholder line each	**Not done**	`docs/adr`	Aum Parmar
☐	P4: the gateway has no routes for each service's API documentation (/api/{service}/v3/api-docs)	**Not done**	`application.yml:121`	Param Patel
☐	A removed member or narrowed role is felt only at the next token refresh (up to five minutes); no service compares a token with current permissions	**Not done**	`platform-defaults.yml:22`	Param Patel
☐	Few tests: gateway has 3 test files, org-service 5	**Partial**	`services/gateway/src/test` `services/org-service/src/test`	Param Patel
☐	L1: the semantic-pointers table has no endpoint	**Partial**	`V1__memory.sql:48`	Love Shah
☐	L2: no episode is written when a run finishes; the orchestrator only calls remember and recall	**Not done**	`MemoryClient.java:74`	Love Shah
☐	The failure path when memory-service is down is untested	**Not done**	`services/memory-service/src/test`	Love Shah
☐	Not verified live on 6 October: in-place progress refresh in Chat	**Needs confirmation**	`Readiness log:292`	Love Shah
☐	The event bus (Kafka) is wired but unused; tasks run over HTTP. A design choice, recorded in the README	**Not done**	`README.md:179`	Love Shah
☐	N2: a workspace cannot set its own approval policy per action class; the flag is fixed to false (outbound and destructive actions still always need approval)	**Not done**	`AgentRunner.java:2448`	Panthil Shah
☐	Not verified live on 6 October: send back with feedback, and the retry endpoint	**Needs confirmation**	`Readiness log:292`	Panthil Shah
☐	A Llama 3.3 70B model asked needless clarifying questions; whether this is fixed is not recorded	**Needs confirmation**	`Readiness log:248`	Panthil Shah
☐	F2: no ingestion-run record is written (the table exists, unused)	**Not done**	`V1__knowledge.sql:102`	Fahim Saiyad
☐	Only uploads are ingested; no connector crawls Drive, Notion, Confluence or a GitHub wiki (the connector package is empty)	**Not done**	`README.md:183`	Fahim Saiyad
☐	Meaning-based search adds nothing until an embedding model is configured (the default sandbox embedder gives random vectors)	**Not done**	`README.md:165`	Fahim Saiyad
☐	I2: rate-limit status is not exposed	**Partial**	`IntegrationController.java:97`	Ismail Mansuri
☐	I3: no row is written to tool_invocations for each tool call	**Not done**	`V1__integrations.sql:42`	Ismail Mansuri
☐	I4: the per-minute rate limit is documented but not applied; only the per-run limit is	**Not done**	`ToolGateway.java:42`	Ismail Mansuri

	Item	Status	Evidence	Area
☐	Disconnect does not revoke the token with the provider	**Not done**	`IntegrationController.java:242`	Ismail Mansuri
☐	Google, Microsoft, Salesforce and Zoom need an OAuth app registered per provider before they connect live; whether any is registered on the live server is not recorded	**Needs confirmation**	`README.md:169`	Ismail Mansuri
☐	Y1: text search and the outcome filter on the audit log still run in the browser	**Partial**	`AuditLog.tsx:40`	Yash Doshi
☐	Y2: TanStack Router and Recharts are still listed although nothing imports them	**Not done**	`package.json:18`	Yash Doshi
☐	Not verified live on 6 October: in-place progress refresh, and announcements naming the agent	**Needs confirmation**	`Readiness log:292`	Yash Doshi
☐	D2: Kubernetes image tag is still a literal ${IMAGE_TAG}, and no CORS origins are set	**Not done**	`gateway.yaml:57`	Afif Momin
☐	D2: the internal service secret is empty; this may be intentional for an external secret store	**Needs confirmation**	`config.yaml:74`	Afif Momin
☐	D3: no CI job builds the container images	**Not done**	`ci.yml`	Afif Momin
☐	CI skips the formatting check (spotless)	**Not done**	`ci.yml:39`	Afif Momin
☐	Thin test coverage: gateway 3, org 5, integrations 5, analytics 5 test files	**Partial**	`gateway tests`	Afif Momin
☐	analytics-service has no owner in the plan or the README	**Needs confirmation**	`analytics-service`	Team
☐	Live site did not respond when checked on 11 October while preparing this report (may be stopped, or the check may have been blocked locally)	**Needs confirmation**	`AWS guide:254`	Afif Momin

4. Team members and roles

Names and roles are **editable fields**: click a field in a PDF reader to change it. They are pre-filled from the Team Contribution Plan (1 October 2026). GitHub accounts are matched to names by module, using the README ownership table.

Member	Name	Role	GitHub account	Section
Member 1	Aum Parmar	Lead, architecture, Platform Core and Contracts	aum2606	Open
Member 2	Param Patel	Backend platform: gateway, auth, organisations	Param2725	Open
Member 3	Love Shah	Agent orchestration and shared memory	LoveShah21	Open
Member 4	Panthil Shah	AI employee agents and agent runtime	PanthilShah	Open
Member 5	Fahim Saiyad	Knowledge base and RAG pipeline	fahim0-3	Open
Member 6	Ismail Mansuri	MCP integration layer (MCP Core)	BitecodesHub	Open
Member 7	Yash Doshi	Frontend (web client)	tempyash007	Open
Member 8	Afif Momin	DevOps, testing and documentation	Afif-Momin	Open

Account matching. Each GitHub account was matched to a name because the README and the plan give the same module to both. The match for *tempyash007* (Yash Doshi) rests on the module and the first name only: **Needs confirmation**.

5. Individual member documentation

Each section follows the same structure: work summary, architecture and workflow, completed work, remaining work and repository links. "Completed" describes the state of the member's **assigned module**. Who wrote the code is stated separately under *Repository record*, from the git history.

Member 1 — Aum Parmar

Role	**Project lead, system architecture, Platform Core and Contracts**
Role source	Combined from the plan ("Project lead and system architecture") and the README ("Platform Core, Contracts"), as agreed for this report.
GitHub account	aum2606
Repository area	docs/, docs/adr/, README.md, platform-core/, platform-contracts/
Plan tasks	A1 **Partial**, A2 **Needs confirmation**, A3 **Done**

Work Summary

- Owns the project's architecture records: decision records (ADRs), the README and the design documents in *docs/*.
- Owns **platform-core**, the library every service imports for settings, request context, errors, encryption of secrets, permission codes and retry presets, and **platform-contracts**, reserved for shared request and response types.
- Contribution to the whole: every service behaves the same way (errors, security, retries) because these rules live in one place.

Architecture and Workflow

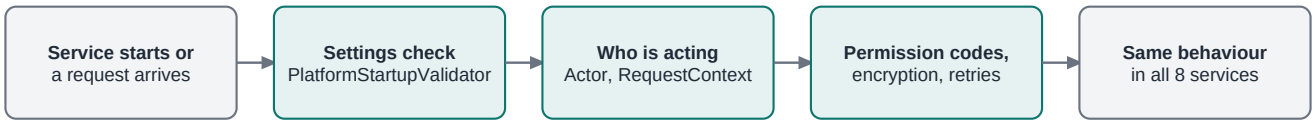


Figure. Aum Parmar: input, processing and output.

- **Input:** a service starting up, and every incoming request.
- **Processing:** refuses unsafe settings (for example a development key in a deployed environment), records who is acting for which workspace, encrypts stored secrets with a per-value key, and provides the fixed list of permission codes.
- **Output:** one error format, encrypted values and permission decisions that all services share. **platform-web** builds on it for token checks and the audit outbox.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	Startup validation, request context, standard errors, envelope encryption, permission codes, resilience presets, run-time settings (code present)	`PlatformStartupValidator` `EnvelopeEncryptionService` `Permission`
☑	Team and module ownership table in the README (plan task A3)	`README.md:279`
☑	Architecture documentation: README architecture section and the file-by-file code guide (reported, dated 4 and 10 October)	`README.md:82` `docs/2026-10-10_Understand-Code_v1.md`

Remaining Work

	Remaining	Status	Evidence
☐	A1: the stack-change ADR does not name the old stack (FastAPI, Next.js), lists TanStack Router, which the web app does not use, and shares number 0001 with another ADR	**Partial**	`docs/adr/0001-record-stack-change.md`

	Remaining	Status	Evidence
☐	A2: confirm each requirement status in the design document against the code. The Word file is unchanged since it was added, so the check cannot be seen in git	**Needs confirmation**	`System Design Document v2`
☐	platform-contracts holds package folders only; there are no shared types yet	**Not done**	`platform-contracts`
☐	README is out of date in places: it says agents do not use memory, five connectors have no live adapter and no endpoint sets a budget; the code now has all three	**Not done**	`README.md:167`
☐	Five ADR files (0001 to 0005 "week" decisions) hold one placeholder line each	**Not done**	`docs/adr`

Repository record

Five commits from *aum2606*. Each adds a three-line placeholder ADR ("Week N architecture update") in *docs/adr/*. No commits to platform-core or platform-contracts come from this account; that code was committed from the *BitecodesHub* account. Individual authorship: **Needs confirmation**.

Relevant Repository Links

- `docs/adr`
- `README: team table`
- `platform-core`
- `platform-contracts`
- `Understand the code (guide)`
- Architecture file: Components and responsibilities
- Architecture file: How components communicate
- Project progress: remaining work · Team table · Contents

Member 2 — Param Patel

Role	**Backend platform: gateway, authentication and organisations**
GitHub account	Param2725
Repository area	services/gateway, services/identity-service, services/org-service
Plan tasks	P1 **Done**, P2 **Done**, P3 **Done**, P4 **Not done**

Work Summary

- Owns the front door and the identity of everyone using the system.
- **Gateway**: routes each request, checks the sign-in token and rate-limits with Redis. **Identity**: users, sign-in, sessions, roles, permission codes and signed tokens. **Organisation**: workspaces, invitations and encrypted API keys.
- Contribution to the whole: every other service trusts the token identity signs, and the orchestrator takes AI provider keys from org-service.

Architecture and Workflow

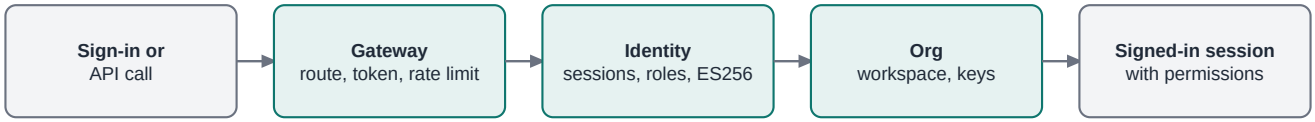


Figure. Param Patel: input, processing and output.

- **Input:** an e-mail and password, or an API call carrying a bearer token.
- **Processing:** identity checks the password, opens a session and signs a five-minute ES256 token listing the person's permissions; it publishes its public keys (JWKS) so every service can verify tokens itself.
- **Output:** a token and a refresh cookie; org-service returns the workspace and stores secrets encrypted. Identity tells the orchestrator when a member leaves, so their schedules pause.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	P1: workspace settings update (PATCH) with a permission and same-workspace check	`WorkspaceController.java:171`
☑	P2: reading a workspace is limited to its members (others get 404)	`WorkspaceController.java:151`
☑	P3: list of signed-in devices and sign-out per device or everywhere	`PasswordController.java:76` `AuthController.java:140`

☑	Sign-in, refresh, sign-out, ES256 tokens with key rotation, JWKS, roles (code present; README reports live verification)	`TokenService`
		`JwksController`
		`RoleController`

☑	Encrypted credential store and invitations	`CredentialService` `InvitationService`
☑	Gateway security, rate-limit keys and a route-table test	`GatewaySecurityConfig` `GatewayRoutesTest`

Remaining Work

	Remaining	Status	Evidence
☐	P4: the gateway has no routes for each service's API documentation (/api/{service}/v3/api-docs)	**Not done**	`application.yml:121`
☐	A removed member or narrowed role is felt only at the next token refresh (up to five minutes); no service compares a token with current permissions	**Not done**	`platform-defaults.yml:22`

	Remaining	Status	Evidence
☐	Few tests: gateway has 3 test files, org-service 5	**Partial**	`services/gateway/src/test` `services/org-service/src/test`

Repository record

Five commits from *Param2725*. Each adds one comment line (for example `// Week 5 update by Param2725`) to *WorkspaceController.java*. No functional code in this module is authored by this account. The implementation listed above was committed from the *BitecodesHub* account (for example commits 9d4121c, 215d358 and 67ccbe0). Individual authorship: **Needs confirmation** by the team.

Relevant Repository Links

- `services/gateway`
- `services/identity-service`
- `services/org-service`
- `Gateway route table`
- Architecture file: How components communicate
- Architecture file: Components and responsibilities
- Project progress: remaining work · Team table · Contents

Member 3 — Love Shah

Role	**Agent orchestration and shared memory**
GitHub account	LoveShah21
Repository area	services/orchestrator-service (orchestration), services/memory-service
Plan tasks	L1 **Partial**, L2 **Not done**, L3 **Done**

Work Summary

- Owns how work is coordinated: Chat routing, turning a request into a goal and tasks, running tasks in the background, the Orchestrator board and Schedules.
- Owns **memory-service**: each agent's notes (remember and recall) and run episodes.
- Contribution to the whole: this is the path every request takes from Chat to an agent, and how agents keep useful facts between runs.

Architecture and Workflow

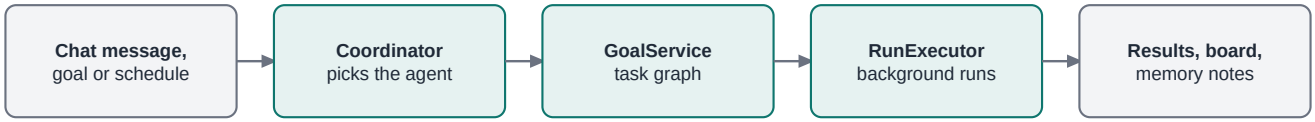


Figure. Love Shah: input, processing and output.

- **Input:** a chat message, a goal from the Orchestrator page, or a schedule that falls due.
- **Processing:** the coordinator chooses an agent (@mention, intent, model plan, keywords, else the General Employee); tasks run as a dependency graph, each earlier result handed to the next agent; steps are saved as they happen.
- **Output:** answers in Chat, a live board of work, and notes an agent can recall next time. Each task is executed by the agent runtime (see Panthil Shah).

Completed Work

	Completed (evidence in the repository)	Evidence
☑	Chat routing with a stated reason, and the General Employee fallback	`CoordinatorService`
☑	Goals as task graphs; background execution on virtual threads	`GoalService` `RunExecutor`
☑	Orchestrator board and Schedules in plain English	`BoardService` `ScheduleParser`
☑	Agent notes: remember, recall, pin, forget; secrets refused	`AgentMemoryService` `SecretGuard` `AgentMemoryTool`
☑	L1 (partly): endpoints to read and write episodes	`MemoryController.java:76`
☑	L3: tests for memory writes and refused or invalid writes	`InternalMemoryControllerTest.java:95`

Remaining Work

	Remaining	Status	Evidence
☐	L1: the semantic-pointers table has no endpoint	**Partial**	`V1__memory.sql:48`
☐	L2: no episode is written when a run finishes; the orchestrator only calls remember and recall	**Not done**	`MemoryClient.java:74`
☐	The failure path when memory-service is down is untested	**Not done**	`services/memory-service/src/test`
☐	Not verified live on 6 October: in-place progress refresh in Chat	**Needs confirmation**	`Readiness log:292`

	Remaining	Status	Evidence
☐	The event bus (Kafka) is wired but unused; tasks run over HTTP. A design choice, recorded in the README	**Not done**	`README.md:179`

Repository record

Five commits from *LoveShah21*. Each adds one comment line (for example `// Week 5 update by LoveShah21`) to *EpisodicMemory.java*. No functional code in this module is authored by this account. The implementation listed above was committed from the *BitecodesHub* account (for example commits 9d4121c, 215d358 and 67ccbe0). Individual authorship: **Needs confirmation** by the team.

Relevant Repository Links

- `orchestrator .../chat`
- `orchestrator .../board`
- `orchestrator .../schedule`
- `services/memory-service`
- Architecture file: Main data flow: a chat request
- Architecture file: Knowledge and memory
- Project progress: remaining work · Team table · Contents

Member 4 — Panthil Shah

Role	**AI employee agents and agent runtime**
GitHub account	PanthilShah
Repository area	services/orchestrator-service (agents, runtime)
Plan tasks	N1 **Done**, N2 **Not done**, N3 **Done**

Work Summary

- Owns the AI employees themselves and the loop that runs them: ask the model, use tools, wait for approval or an answer, continue.
- Includes agent versions and templates, tool grants, approvals, clarifying questions, send-back with feedback and run retry.
- Contribution to the whole: this is where the safety promise is kept: risky actions stop and wait for a person.

Architecture and Workflow

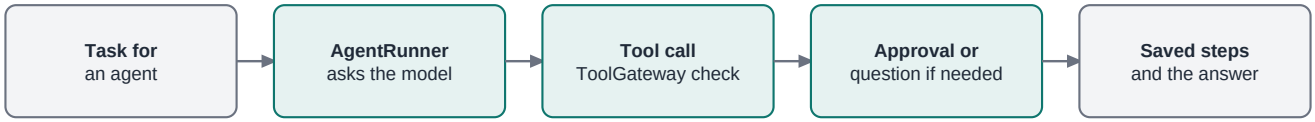


Figure. Panthil Shah: input, processing and output.

- **Input:** a task with the agent's instructions, tools and conversation so far.
- **Processing:** AgentRunner asks the model through llm-core; each tool call goes through mcp-core's ToolGateway. Sending, posting or deleting parks the run for approval; a question to a person parks it until answered. An approved call runs exactly once on resume.
- **Output:** a run trace step by step, approvals in the inbox, and the final answer.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	N1: an approval records the real action class (read, write, outbound, destructive)	`AgentRunner.java:2628` `ApprovalService.java:238`
☑	N3: endpoints to add and remove an agent's tool grants	`AgentGrantController.java:83`
☑	Clarifying questions mid-run; General Employee fallback; agent templates	`QuestionService` `GeneralEmployee` `AgentTemplates`
☑	Send back with feedback (two-round cap) and run retry	`RunRetryController`

Remaining Work

	Remaining	Status	Evidence
☐	N2: a workspace cannot set its own approval policy per action class; the flag is fixed to false (outbound and destructive actions still always need approval)	**Not done**	`AgentRunner.java:2448`
☐	Not verified live on 6 October: send back with feedback, and the retry endpoint	**Needs confirmation**	`Readiness log:292`
☐	A Llama 3.3 70B model asked needless clarifying questions; whether this is fixed is not recorded	**Needs confirmation**	`Readiness log:248`

Repository record

Five commits from *PanthilShah*. Each adds one comment line (for example `// Week 5 update by PanthilShah`) to *OrchestratorApplication.java*. No functional code in this module is authored by this account. The implementation listed above was committed from the *BitecodesHub* account (for example commits 9d4121c, 215d358 and

67ccbe0). Individual authorship: **Needs confirmation** by the team.

Relevant Repository Links

- `AgentRunner.java`
- `ApprovalService.java`
- `orchestrator .../web (REST controllers)`
- `llm-core ModelRouter.java`
- Architecture file: Tool calls and approvals
- Architecture file: Main data flow: a chat request
- Project progress: remaining work · Team table · Contents

Member 5 — Fahim Saiyad

Role	**Knowledge base and RAG pipeline**
GitHub account	fahim0-3
Repository area	services/knowledge-service
Plan tasks	F1 **Done**, F2 **Not done**, F3 **Done**

Work Summary

- Owns the company knowledge base: upload documents, split them into passages, embed them, and search them so agents answer with citations (retrieval-augmented generation, RAG).
- Includes restricted sources and each agent's private documents.
- Contribution to the whole: grounded, cited answers, and an honest "no document supports an answer" when nothing matches.

Architecture and Workflow

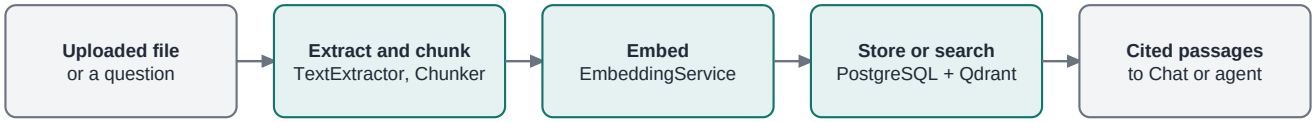


Figure. Fahim Saiyad: input, processing and output.

- **Input:** a file upload, or a search from Chat or an agent's knowledge tool.
- **Processing:** text is extracted, split into heading-aware passages and embedded; search runs keyword and meaning search, fuses the two by rank, applies a relevance floor and filters by what the caller may read.
- **Output:** ranked passages with citations. If Qdrant or the embedding model is missing, it falls back to keyword search and says so. Embeddings are made through the orchestrator's model router.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	F1: query embeddings use the provider and model each source was built with	`RetrievalService.java:369`
☑	F3: tests for query phrasing and the relevance floor	`RetrievalQueryTest.java:15`
☑	Upload, replace, reindex, rename and delete; heading-aware chunking	`IngestionService` `Chunker`
☑	Hybrid search with rank fusion and access filtering; change of embedding model rebuilds every source	`RetrievalService` `EmbeddingModelChange`

Remaining Work

	Remaining	Status	Evidence
☐	F2: no ingestion-run record is written (the table exists, unused)	**Not done**	`V1__knowledge.sql:102`
☐	Only uploads are ingested; no connector crawls Drive, Notion, Confluence or a GitHub wiki (the connector package is empty)	**Not done**	`README.md:183`
☐	Meaning-based search adds nothing until an embedding model is configured (the default sandbox embedder gives random vectors)	**Not done**	`README.md:165`

Repository record

Five commits from *fahim0-3*. Each adds one comment line (for example `// Week 5 update by fahim0-3`) to *EmbeddingService.java*. No functional code in this module is authored by this account. The implementation listed above was committed from the *BitecodesHub* account (for example commits 9d4121c, 215d358 and 67ccbe0).

Individual authorship: **Needs confirmation** by the team.

Relevant Repository Links

- `services/knowledge-service`
- `knowledge .../service`
- `KnowledgeController.java`
- `README: why the query is written this way`
- Architecture file: Knowledge and memory
- Project progress: remaining work · Team table · Contents

Member 6 — Ismail Mansuri

Role	**MCP integration layer (MCP Core)**
GitHub account	BitecodesHub
Repository area	mcp-core, services/integrations-service
Plan tasks	I1 **Done**, I2 **Partial**, I3 **Not done**, I4 **Not done**

Work Summary

- Owns how agents reach business tools through the Model Context Protocol (MCP): one tool gateway, and one adapter per connector.
- Includes **integrations-service**: connect, test and disconnect accounts, OAuth sign-in for Google, Microsoft and Salesforce, and handing tokens to the orchestrator.
- Contribution to the whole: agents can act on 19 tools, live or with practice data, and every call is checked the same way.

Architecture and Workflow

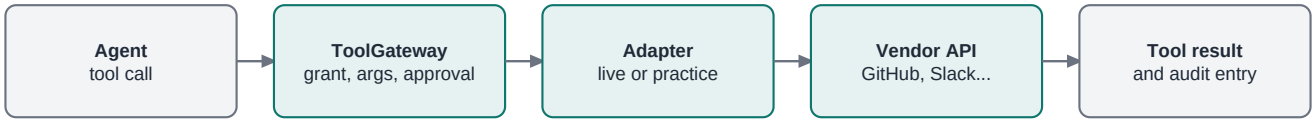


Figure. Ismail Mansuri: input, processing and output.

- **Input:** a tool call from an agent run, with arguments.
- **Processing:** ToolGateway checks the grant and scope, validates arguments against the tool's schema, parks risky calls for approval, applies the per-run limit and a circuit breaker. The adapter calls the vendor with a token fetched from integrations-service, or answers from practice data.
- **Output:** the tool result to the agent; connection health on the Connectors page.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	One tool gateway for every call; argument validation	`ToolGateway` `ArgumentValidator`

☑	Catalogue of 19 connectors with a live adapter for each, plus practice-data adapters	`ConnectorCatalog`
		`mcp-core .../live`
		`mcp-core .../sandbox`

☑	OAuth for Google, Microsoft and Salesforce connectors	`OAuthProviders` `OAuthService`
☑	I1: disconnect clears the stored credential reference	`IntegrationController.java:242`
☑	I2 (partly): connection health (status, last error, last checked) is exposed	`IntegrationController.java:97`
☑	GitHub repository tools for agents (read, browse, write to a branch)	`GitHubAdapter`

Remaining Work

	Remaining	Status	Evidence
☐	I2: rate-limit status is not exposed	**Partial**	`IntegrationController.java:97`
☐	I3: no row is written to tool_invocations for each tool call	**Not done**	`V1__integrations.sql:42`
☐	I4: the per-minute rate limit is documented but not applied; only the per-run limit is	**Not done**	`ToolGateway.java:42`
☐	Disconnect does not revoke the token with the provider	**Not done**	`IntegrationController.java:242`

	Remaining	Status	Evidence
☐	Google, Microsoft, Salesforce and Zoom need an OAuth app registered per provider before they connect live; whether any is registered on the live server is not recorded	**Needs confirmation**	`README.md:169`

Repository record

BitecodesHub authored 27 of the 67 commits on *main*, including the main implementation commits across every module (for example 67ccbe0, 1,101 files; 215d358, 779 files; 110ff57, 92 files). Five of the 27 are "platform-core: week N" commits that each add one comment line to PlatformProperties.java. The account is matched to Ismail Mansuri because the README gives MCP Core to *BitecodesHub* and the plan gives the MCP integration layer to Ismail Mansuri.

Relevant Repository Links

- `mcp-core`
- `services/integrations-service`
- `ToolGateway.java`
- `Live adapters folder`
- Architecture file: Tool calls and approvals
- Architecture file: Technologies and integrations
- Project progress: remaining work · Team table · Contents

Member 7 — Yash Doshi

Role	**Frontend (web client)**
GitHub account	tempyash007
Repository area	web/
Plan tasks	Y1 **Partial**, Y2 **Not done**, Y3 **Done**

Work Summary

- Owns the web app: every screen people use, from Chat and the Orchestrator board to Knowledge, Connectors, Approvals, Audit log, Analytics and Settings, plus the public /home and /trust pages.
- Includes the API client, session and permission handling, the design system and its tests.
- Contribution to the whole: makes the platform usable by non-technical people, with buttons shown only to roles allowed to use them.

Architecture and Workflow

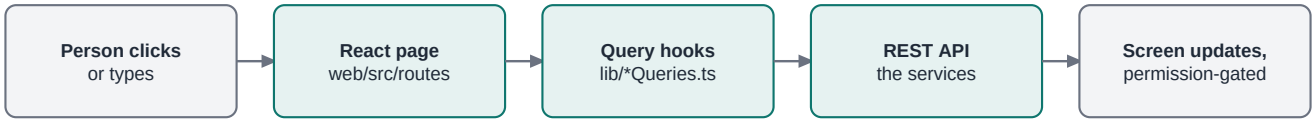


Figure. Yash Doshi: input, processing and output.

- **Input:** clicks and typing in the browser.
- **Processing:** each page uses TanStack Query hooks; one HTTP client adds the token, refreshes it on 401 and turns errors into plain words; session.ts decides which actions a role may see.
- **Output:** screens that update as runs progress, with permission-gated actions.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	27 page files (22+ signed-in screens, public /home and /trust)	`web/src/routes`
☑	One HTTP client with token refresh; session and permission checks	`api.ts` `session.ts`
☑	Y3: signed-in devices and sign-out on the Profile page	`Profile.tsx:181`
☑	Y1 (partly): audit-log action, actor, resource and date filters run on the server	`queries.ts:2597`
☑	Design-system rules tested; 150 web test files; accessibility audit reported (README)	`design-system.test.ts`

Remaining Work

	Remaining	Status	Evidence
☐	Y1: text search and the outcome filter on the audit log still run in the browser	**Partial**	`AuditLog.tsx:40`
☐	Y2: TanStack Router and Recharts are still listed although nothing imports them	**Not done**	`package.json:18`
☐	Not verified live on 6 October: in-place progress refresh, and announcements naming the agent	**Needs confirmation**	`Readiness log:292`

Repository record

Five commits from *tempyash007*. Each adds one comment line (for example `// Week 5 update by tempyash007`) to *AuditLog.tsx*. No functional code in this module is authored by this account. The implementation listed above was committed from the *BitecodesHub* account (for example commits 9d4121c, 215d358 and 67ccbe0). Individual authorship: **Needs confirmation** by the team. The match of this account to Yash Doshi is also **Needs**

confirmation.

Relevant Repository Links

- `web/`
- `web/src/routes`
- `web/src/components`
- `web/src/lib`
- Architecture file: Main data flow: a chat request
- Architecture file: Ways to run it
- Project progress: remaining work · Team table · Contents

Member 8 — Afif Momin

Role	**DevOps, testing and documentation**
GitHub account	Afif-Momin
Repository area	.github/, infra/, tests
Plan tasks	D1 **Done**, D2 **Not done**, D3 **Not done**, D4 **Done**

Work Summary

- Owns how the system is built, tested and shipped: CI on GitHub Actions, containers, Compose, the one-click launcher, Kubernetes manifests, monitoring and the AWS deployment.
- Also owns test coverage across services.
- Contribution to the whole: every push is built and tested, and the live server follows *main* with automatic rollback.

Architecture and Workflow

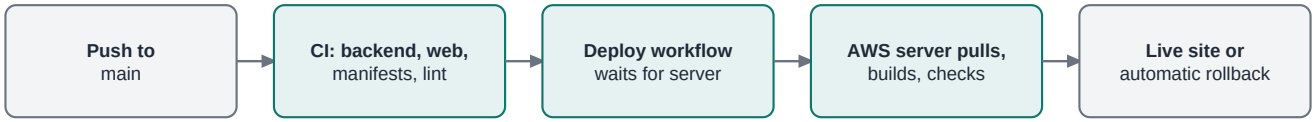


Figure. Afif Momin: input, processing and output.

- **Input:** a push to *main*.
- **Processing:** CI runs the backend build and tests, the web build, tests and lint, and checks the Kubernetes manifests. The AWS server polls *main* every two minutes, builds the images itself and health-checks them.
- **Output:** a deployed commit, or the previous version restored automatically; deploy status at /deploy-status.json.

Completed Work

	Completed (evidence in the repository)	Evidence
☑	D1: web lint runs in CI	`ci.yml:90`
☑	D4: the five services that had no tests on 2 October now have tests	`analytics tests` `integrations tests`

☑	Containers, Compose stack, one-click launcher, Prometheus and Grafana	`Dockerfile`
		`compose`
		`launcher`

☑	AWS pull deployment with health check and rollback; deploy workflow	`cloudformation.yaml` `deploy.yml`

Remaining Work

	Remaining	Status	Evidence
☐	D2: Kubernetes image tag is still a literal ${IMAGE_TAG}, and no CORS origins are set	**Not done**	`gateway.yaml:57`
☐	D2: the internal service secret is empty; this may be intentional for an external secret store	**Needs confirmation**	`config.yaml:74`
☐	D3: no CI job builds the container images	**Not done**	`ci.yml`
☐	CI skips the formatting check (spotless)	**Not done**	`ci.yml:39`
☐	Thin test coverage: gateway 3, org 5, integrations 5, analytics 5 test files	**Partial**	`gateway tests`

Repository record

Ten commits from *Afif-Momin*: five, then the same five again with a corrected e-mail address. Each adds one comment line (for example "# Week 5 CI update by Afif-Momin") to *ci.yml*. The CI, infrastructure and test work

listed above was committed from the *BitecodesHub* account. Individual authorship: **Needs confirmation**.

Relevant Repository Links

- `.github/workflows`
- `infra`
- `infra/k8s`
- `AWS deployment guide`
- Architecture file: Ways to run it
- Architecture file: Technologies and integrations
- Project progress: remaining work · Team table · Contents

6. About this report

6.1 How the evidence was gathered

- Read the README, the code guide, the readiness log, the AWS guide, the ADRs and the Team Contribution Plan.
- Checked all 27 plan tasks against the current code, recording a file and line for each; the key results were re-checked by hand.
- Counted source, test and migration files per module, and mapped service-to-service calls from the code.
- Read the git history: authors, dates and what each commit changes.

6.2 Tests

No tests were run for this report. The last recorded full run is in the readiness log (6 October 2026): Maven build green; orchestrator 1,112 tests, llm-core 107, memory 36, knowledge 91, identity 70, platform-core 33; web type check, lint and build green; 1,306 web tests in 110 files. Later commits added tests (there are now 150 web test files), so these figures predate the current code.

6.3 Plan task status

Done: 13, **Partial**: 4, **Not done**: 9, **Needs confirmation**: 1 (27 tasks from the 1 October plan).

Task	Member	Status
A1	Aum Parmar	Partial
A2	Aum Parmar	Needs confirmation
A3	Aum Parmar	Done
P1	Param Patel	Done
P2	Param Patel	Done
P3	Param Patel	Done
P4	Param Patel	Not done
L1	Love Shah	Partial
L2	Love Shah	Not done
L3	Love Shah	Done
N1	Panthil Shah	Done
N2	Panthil Shah	Not done
N3	Panthil Shah	Done

Task	Member	Status
F1	Fahim Saiyad	Done
F2	Fahim Saiyad	Not done
F3	Fahim Saiyad	Done
I1	Ismail Mansuri	Done
I2	Ismail Mansuri	Partial
I3	Ismail Mansuri	Not done
I4	Ismail Mansuri	Not done
Y1	Yash Doshi	Partial
Y2	Yash Doshi	Not done
Y3	Yash Doshi	Done
D1	Afif Momin	Done
D2	Afif Momin	Not done
D3	Afif Momin	Not done
D4	Afif Momin	Done

6.4 Repository history notes

Commits by account (67 commits on *main*), from `git log` on 11 October 2026:

Account	Commits	What the commits change
BitecodesHub	27	22 implementation and documentation commits across services, libraries, web, infra and docs, including 9d4121c ("add meaningful changes for all team members"); 5 add one comment line each to PlatformProperties.java
Afif-Momin	10	One comment line each in ci.yml (five repeated with a corrected e-mail address)
aum2606	5	Three-line placeholder ADRs

Account	Commits	What the commits change
Param2725, LoveShah21, PanthilShah, fahim0-3, tempyash007	5 each	One comment line each in a single file of their module

Dates. 45 "week N" commits (the 40 from the seven team accounts and five from *BitecodesHub*) carry dates from 31 August to 4 October 2026, each at exactly 00:00, yet in the history they all come after a commit dated 30 September 2026. Their dates therefore do not show when they were made. The plan itself asks that dates be real; the team may wish to address this before the history is reviewed.

6.5 Links in this document

- Repository links open github.com/BitecodesHub/AI_Workforce_OS at *main*. Every linked path was checked to be tracked by git before this PDF was written. Line numbers reflect the code on 11 October 2026 and may drift.
- The architecture file and this PDF are new and not yet committed. Their GitHub links return "not found" until both are committed and pushed. Local copies: `docs/2026-10-11_Architecture_v1.md` and `docs/2026-10-11_Project-Documentation_v1.pdf`.
- Internal links (contents, member names, "Back to contents") work in any PDF reader. Editable name and role fields need a reader with form support.